

RECURSION

THINGS YOU SHOULD ALREADY KNOW FROM PRE-CLASS WORK

- **Recursion** is when a function calls itself to do its work.
- A **recursive call** is what happens when a function calls itself; the input to the recursive call should be a "smaller" version of the problem.
 - e.g. `multiply(3, 4)` can be evaluated as `4 + multiply(2, 4)`
- A **base case** is an input to a recursive function that does not require another recursive call.
 - Multiplication when one of the inputs is 1
 - Searching when the list is empty

RECURSIVE PROBLEMS ARE PROBLEMS DEFINED IN TERMS OF THEMSELVES!

- Data representation:
 - **JSON** features dictionaries whose keys point to values that are dictionaries whose keys point to values that are dictionaries whose keys point to values that are...
 - **XML** is a format where we place tags inside of tags inside of tags inside of tags inside of...
- Algorithms like **searches** (linear, binary, depth-first, breadth-first, etc.)

Recursion is NOT just something Computer Scientists made up to torture students!

It crops up in...

- how we produce language
- how organisms grow
- (yes, yes...) math & logic & philosophy

AN EXAMPLE

What is a **sentence**?

AN EXAMPLE

What is a **sentence**? A sentence is a sequence of words.

AN EXAMPLE

What is a **sentence?** A sentence is a sequence of words.

phony monkey toy money lose end soda hands ??

AN EXAMPLE

What is a **sentence**? A sentence is a sequence of words following some set of rules (a **syntax**).

- *Don't come running to me!*
- *Homework 6 is due on Wednesday.*
- *Nina isn't listening and she's seen into the mouth of what is really in the fountain and she's seen all the good pigeon-like people shot down and bones be kicked to powder by the insane wild horses.*

(yes, these are sentences—the meaning (**semantics**) is a separate problem)

SENTENCES & CONTEXT-FREE GRAMMARS

We can use the concept of a **context-free grammar** (CFG) to model the rules for what a sentence is.

A CFG provides a set of rules for what kinds of sequences of **symbols** are "valid". Here, think "symbol = word".

- some rules lead to other rules (called "nonterminals")
- other rules lead to the symbols (called "terminals")



A (simple) sentence consists of a noun phrase, a verb phrase, and a noun phrase.

"A big green dog ate my homework."

WORDS	PART OF SENTENCE
A big green dog	noun phrase
ate	verb phrase
my homework.	noun phrase

<sentence> -> <noun phrase> <verb phrase> <noun phrase>

A (simple) sentence consists of a noun phrase, a verb phrase, and a noun phrase.
A noun phrase consists of an adjective phrase and a noun.

WORDS	PART OF NOUN PHRASE
A big green	adjective phrase
dog	noun

WORDS	PART OF NOUN PHRASE
my	adjective phrase
homework	noun

<sentence> -> <noun phrase> <verb phrase> <noun phrase>
<noun phrase> -> <adj phrase> <noun>

A (simple) sentence consists of a noun phrase, a verb phrase, and a noun phrase.
A noun phrase consists of an adjective phrase and a noun.
A noun is one of several words. *(This is a rule that leads to only terminals—no more expansion possible!)*

WORDS	PART OF NOUN PHRASE
A big green	adjective phrase
dog	noun <i>(no need to drill down further here...)</i>

<sentence> -> <noun phrase> <verb phrase> <noun phrase>

<noun phrase> -> <adj phrase> <noun>

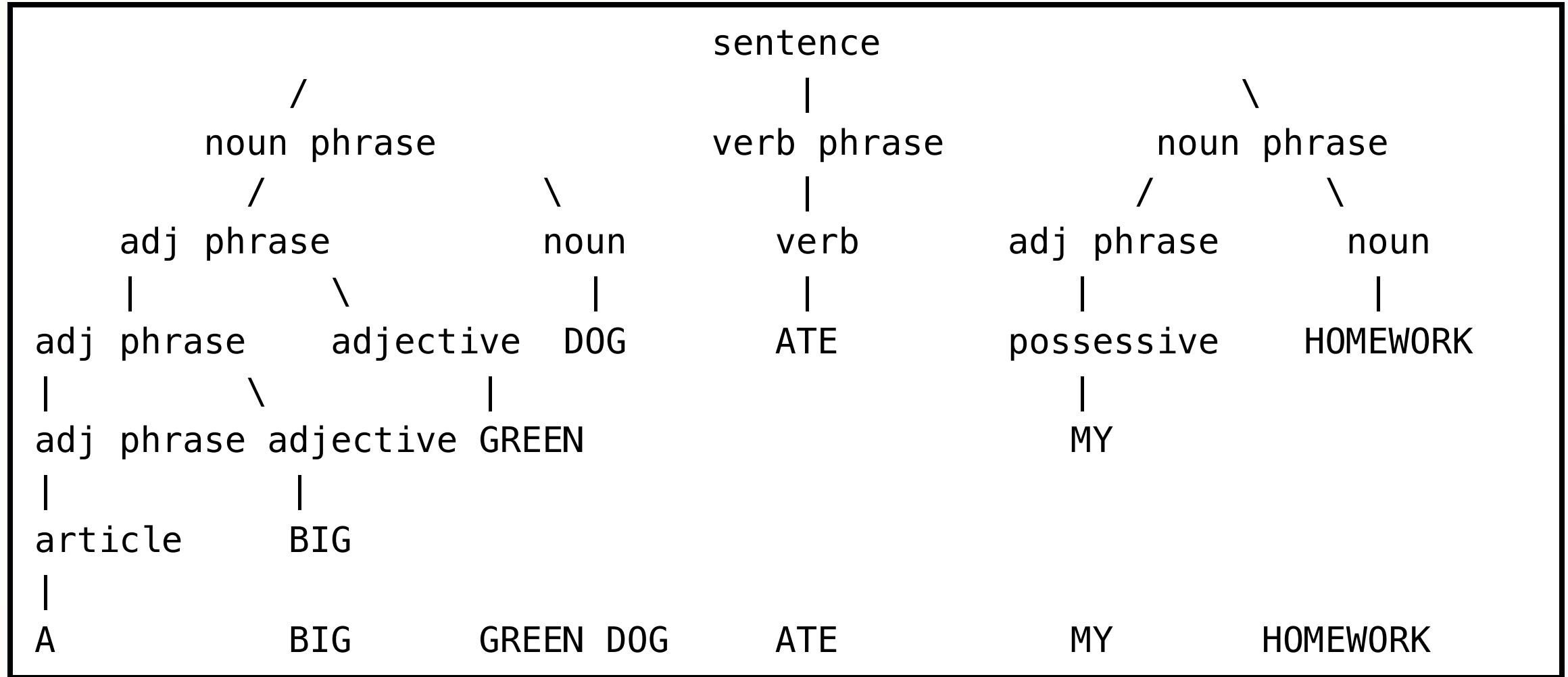
<noun> -> dog | trousers | daughter | nose | homework | wizard | pony

An adjective phrase is an article, or a possessive, or an adjective phrase followed by an adjective. !! !! !! !! !! !!

WORDS	PART OF ADJECTIVE PHRASE
A big	adjective phrase
green	adjective

```
<sentence> -> <noun phrase> <verb phrase> <noun phrase>
<noun phrase> -> <adj phrase> <noun>
<noun> -> dog | trousers | daughter | nose | homework | wizard | pony
<adj phrase> -> <article> | <possessive> | <adj phrase> <adjective>
<article> -> the | a | some | every | that
<possessive> -> <noun phrase>'s | my | your | his | her
<adjective> -> friendly | furious | fragrant | green | severed | little
```

Sentences are built using *rules that refer to themselves recursively.*



"A noun phrase has an adjective phrase which has an adjective phrase which has an adjective phrase which has an article, 'A'."

<sentence> -> <noun phrase> <verb phrase> <noun phrase>

<noun phrase> -> <adj phrase> <noun>

<adj phrase> -> <article> | <possessive> | <adj phrase> <adjective>

<verb phrase> -> <verb> | <adverb> <verb phrase>

<noun> -> dog | trousers | daughter | nose | homework | wizard | pony

<article> -> the | a | some | every | that

<possessive> -> <noun phrase>'s | my | your | his | her

<adjective> -> friendly | furious | fragrant | green | severed | little

<verb> -> ate | found | wrote | killed | mangled | saved | broke

<adverb> -> squarely | incompetently | barely | sort of | awkwardly | to

To get a valid simple sentence, start with a **<sentence>** and repeatedly expand the leftmost non-terminal using these rules.

Here's how we got "A big green dog ate my homework."

<sentence>

<noun phrase> <verb phrase> <noun phrase>

<adj phrase> <noun> <verb phrase> <noun phrase>

<adj phrase> <adj> <noun> <verb phrase> <noun phrase>

<adj phrase> <adj> <adj> <noun> <verb phrase> <noun phrase>

<article> <adj> <adj> <noun> <verb phrase> <noun phrase>

A <adj> <adj> <noun> <verb phrase> <noun phrase>

A big <adj> <noun> <verb phrase> <noun phrase>

A big green <noun> <verb phrase> <noun phrase>

A big green dog <verb phrase> <noun phrase>

A big green dog <verb> <noun phrase>

A big green dog ate <noun phrase>

A big green dog ate <adj phrase> <noun>

A big green dog ate <possessive> <noun>

A big green dog ate my <noun>

A big green dog ate my homework.

Fun game: generate random sentences by choosing a random option whenever expanding with a rule.

<sentence> -> <noun phrase> <verb phrase> <noun phrase>

<noun phrase> -> <adj phrase> <noun>

<adj phrase> -> <article> | <possessive> | <adj phrase> <adjective>

<verb phrase> -> <verb> | <adverb> <verb phrase>

<noun> -> dog | knight | nose | homework | wizard | pony

<article> -> the | a | some | every | that

<possessive> -> <noun phrase>'s | my | your | his | her | their

<adjective> -> friendly | furious | fragrant | green | severed | little

<verb> -> ate | found | wrote | killed | mangled | saved | broke

<adverb> -> squarely | incompetently | barely | sort of | awkwardly | to

 Back To Practice! 

TRACING AND RECURSION(?)

Work together, we'll come around. It will help to write down all of the different calls you need to keep track of.

```
def cat(x):  
    if x > 10:  
        return x  
    return dog(x * 2)
```

```
def dog(y):  
    if y > 10:  
        return y  
    return cat(y * 2)
```

S7 What does `cat(2)` return?

S8 What does `cat(-1)` return?

DON'T LOSE SIGHT OF WHAT YOU KNOW!

Today's topic (recursion) is tricky, but it's manageable if you keep in mind everything you know about calling functions.

1. Functions are called by their names with inputs passed in
2. Executing a function call creates a new scope in which only **input variables** and **variables defined within the function** are accessible
3. The only way to have some information "escape" the body of a function is to **return it**.
4. Other than producing a value, `return` also stops the current function execution and brings us back to the place at which the function was called.

RECURSIVE THINKING

A function is recursive if it invokes itself to do part of its work.

Recursion is a problem-solving approach that can be used to generate simple solutions to certain kinds of problems that are difficult to solve by other means.

Recursion reduces a problem into one or more simpler versions of itself.

RECURSION

An alternate to using loops for solving problems

The core of recursion is taking a big task and breaking it up into a series of related small tasks.

- Example: handing out papers for an exam
 - Iterative: have a TA walk down a row of students, giving each person an exam
 - Recursive: A student takes one exam, pass the rest down the aisle
- Example: Which row are you in?

ANATOMY OF A RECURSIVE FUNCTION

Every recursive function needs at least one **base case** and at least one **recursive part**.

The **base case**:

- handles a simple input that can be solved without resorting to a recursive call.
Can also be thought of as the case where we "end" our recursion.

The **recursive part**:

- contains one or more recursive calls to the function.
- In every recursive call, the parameters must be in some sense "closer" to the base case than those of the original call

PRACTICE

In mathematics, a factorial (denoted by $!$) is the operation that computes the product of all smaller positive integers.

$$n! = n \times (n - 1) \times (n - 2) \times \cdots \times 1$$

`fact(0)` is 1, `fact(1)` is 1, `fact(6)` is $6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$

We want to write a recursive function to calculate `fact(n)`

(L11) What are the base case(s) and recursive calls? (e.g. when do we recurse, when do we not).

PRACTICE

In mathematics, a factorial (denoted by $!$) is the operation that computes the product of all smaller positive integers.

$$n! = n \times (n - 1) \times (n - 2) \times \cdots \times 1$$

`fact(0)` is 0, `fact(1)` is 1, `fact(6)` is $6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$

We want to write a recursive function to calculate `fact(n)`

(C12) Write the function:

```
def fact(N: int) -> int:  
    # TODO
```

RANGE OF A RECURSIVE FUNCTION

An important thing to think about when designing recursive functions is thinking about:

- Where we start with the problem
- the little bit of work that is done on each step
- the end of the problem

What were each of these things for `def fact(N)`?

What about `def print_stars(N)` (from the videos)?

```
def foo(N):  
    if N <= 1:  
        return 0  
    return 1 + foo(N // 2) # Hint: // is integer division  
  
def fizz(N):  
    if N == 0:  
        return 0  
    return N % 10 + fizz(N // 10)
```

What do these print?

- **S8** `print(foo(16))`
- **S9** `print(fizz(1100))`



PRACTICE:

Consider we want to write the function `remove_blanks(word)` that takes in a string and returns the same string without any spaces in it.

So `remove_blanks("phil a del phia")` returns `"philadelphia"`

Before writing any code... (L13)

- What is the base case? (Why is it the empty string? `" "`)
- What is the recursive case?
- What is the work done on each step?

PRACTICE: (C14)

Finish writing this function:

```
def remove_blanks(word: str) -> str:  
    # takes in a string and returns the same string without any spaces  
    # TODO: What do you put here?
```

PRACTICE:

We want to write a function `sum_numbers` that gets the sum of all integers in a list recursively. Do this in **C16**

First think:

- What are the base case(s) ?
- What is the little bit of work done on each step?
- What do we tell the function to do recursively?

```
def sum_numbers(nums_list):  
    # TODO  
    # return a sum of all the numbers in the list  
    # your soln doesn't have to be perfect, remember that this is practi
```

NEXT TIME/EXTRAS

PRACTICE: (L11)

We want to write the function `ping_pong(N)` which prints "ping" and then "pong" in alternating order for a total of N prints.

<code>ping_pong(3)</code>	prints:	<code>ping</code>	then	<code>pong</code>	then	<code>ping</code>
<code>ping_pong(2)</code>	prints:	<code>ping</code>	then	<code>pong</code>		
<code>ping_pong(1)</code>	prints:	<code>ping</code>				

(L11) Does this code work? Why or why not:

```
def ping_pong(N):  
    if N <= 0:  
        return  
    if N % 2 == 1:  
        print("ping")  
    else:
```


HELPER FUNCTIONS

Sometimes to do recursion we need to remember a bit more information than is provided to the overall problem.

In this case, what other information do we need for `ping_pong(N)` to get a working recursive solution?

Why can't we just recursively call `ping_pong(N-1)`?

PRACTICE (C12)

We want to write the function `ping_pong(N)` which prints "ping" and then "pong" in alternating order for a total of N prints.

<code>ping_pong(3)</code>	prints:	<code>ping</code>	then	<code>pong</code>	then	<code>ping</code>
<code>ping_pong(2)</code>	prints:	<code>ping</code>	then	<code>pong</code>		
<code>ping_pong(1)</code>	prints:	<code>ping</code>				

(C12) finish writing the fixed version:

```
def ping_pong_helper(N, extra):  
    # TODO: do something here  
  
def ping_pong(N):  
    ping_pong_helper(N, _____) # You probably want to pass in either 0
```

PRACTICE:

Consider we want to write the function `find_factors(N)` that returns a set containing all positive factors of the input integer `N`.

A number `x` is a *factor* of `N` if and only if `N % x == 0`.

NOTE: This is a different problem than one you will see on the homework called `gcd`

Before writing any code **(L13)**

- What is the base case?
- What is the recursive case?
- What is the work done on each step?

HELPER FUNCTIONS

Sometimes to do recursion we need to remember a bit more information than is provided to the overall problem.

In this case, what other information do we need for `find_factors(N)` to get a working recursive solution?

Why can't we just recursively call `find_factors(N-1)`?

PRACTICE: (C14)

Finish writing this function:

```
def find_factors_helper(current, N):  
    # TODO: What do you put here?  
  
def find_factors(N):  
    return find_factors_helper(0, N)
```

PRACTICE:

Consider we want to write the **recursive** function `binary_search(lst, target)` that returns the position of `target` in the `lst` or `-1` if it's not present.

Before writing any code **(L15)**

- What is the base case(s)?
- What is the recursive case?
- What is the work done on each step?
- What additional information needs to be passed in to do a Binary Search/what's the signature of the helper function?

HELPER FUNCTIONS

Binary Search works by repeatedly shrinking the *feasible search area*.

- `lo` is the left-most/lowest index that might still hold the target
- `hi` is the right-most/highest index that might still hold the target
- want to keep searching until we find our target or run out of space to search:
when `lo` moves to the right of `hi`

PRACTICE: (C14)

Finish writing this function:

```
def binary_search(lst: list[int], target: int) -> int:
    return binary_search_helper(lst, target, _____)
def binary_search_helper(lst: list[int], target: int, lo: int, hi: int)
    # TODO!
```